Warehouse Wizard × NetSuite — Integration Proposal

Sep 22, 2026 · @Russell Hunte

Receiving moves onto the warehouse floor in Warehouse Wizard and posts straight into NetSuite, ending the double entry Simplex does today. All of it can be built. One part — completing a receipt against an inbound shipment — needs a small piece of code deployed inside NetSuite that was not identified during the meeting, and that is the most important finding in this document. Phase 1, which solves the double-entry problem on its own, is estimated at 60 to 96 working days.

Compiled from the integration meeting of 21 September 2026 between Roget Williams, Wilson Reyes, Jan Roel and Russell Hunte (Classic IT), a review of Oracle's published NetSuite documentation, and an audit of the Warehouse Wizard codebase.

How to read this

This document is written for four readers. No programming knowledge is needed anywhere except the clearly marked notes for the NetSuite developer at the end.

Reader	Read	Time
Roget Williams (management and finance)	Summary, Decisions, Possible, Cost, Comparison, Delivery, Open questions, Recommendation	20 min
The CFO (Roget Williams)	Summary, Cost, Comparison, Recommendation	12 min
Wilson Reyes (inventory systems analyst)	Summary, Decisions, Qualification, Possible, Receiving, Harder questions, Open questions	25 min
Jan Roel (NetSuite developer)	Summary, Possible, Receiving, Harder questions, Delivery, Open questions, Developer notes	25 min
Everyone	Executive summary and Open questions	6 min

Three labels are used throughout and they mean exactly what they say. **Confirmed** means verified in Oracle's published documentation, observed directly in the Warehouse Wizard code, or demonstrated on screen during the meeting. **Expected** means standard NetSuite

behaviour that is very likely true for Simplex but has not been checked against Simplex's own account settings. **Unverified** means genuinely unknown, and every such item is listed in Open questions with an owner and priced as a risk.

Executive summary

The problem

Every delivery is received twice today. Once into Warehouse Wizard by the warehouse team, and once into NetSuite by someone in the office. This is double handling, it is slow, and each duplicate entry is a chance for the two systems to disagree about what is actually in the building.

Roget put the objective plainly: "My aim is to try to eliminate duplicating processes. If there's a way that we could hit the two systems with one entry, that would be great."

The decision reached

Goods are received **once, in Warehouse Wizard, on the warehouse floor**, and Warehouse Wizard tells NetSuite what was received. NetSuite stays the system of record for stock, cost and financials. Warehouse Wizard becomes the hands and eyes of the warehouse feeding it.

Roget: "The item receipt should really happen in Wizard and then NetSuite should read that this PO number was received." Wilson: "Actually that was the ideal process for me."

Is it possible?

Yes, with one exception that must be resolved before work starts.

Receiving against purchase orders and transfer orders uses standard, documented NetSuite integration points that need nothing installed in NetSuite. Sales order picking, returns and transfers likewise.

The exception is **inbound shipments** — the document Simplex uses for every imported container, and the one Wilson specifically asked for. NetSuite's standard integration can read an inbound shipment perfectly well, but provides **no standard way to record a receipt against one**. Oracle documents that action only as something a script running inside NetSuite can do. A small piece of code must therefore be written and deployed inside NetSuite by Jan, roughly two to four days of his time, before Warehouse Wizard can complete an inbound shipment receipt.

This is not a blocker. It is a bounded piece of work that has to be scheduled. But it was not identified during the meeting, and if it is not planned for, the single most important receiving path for Simplex would fail late in the project.

What it costs

The scope discussed is substantially larger than the original Warehouse Wizard engagement. Russell flagged this during the meeting: "the scope is changing a bit, it's going to look like a serious rework of items which were originally discussed and then said it wasn't needed."

The work divides into three phases that can be approved and funded independently.

Phase	What it delivers	Estimated effort	Recommendation
Phase 1 — Inbound	One-entry receiving against purchase orders, inbound shipments and transfer orders. Pine and Lower Estate brought into Warehouse Wizard	60–96 days	**Approve now.** This is where the return is
Phase 2 — Outbound	Sales order picking from NetSuite, units of measure, partial pallets, picking-area visibility, scan verification, transfer despatch	58–92 days	Approve in principle, schedule after Phase 1 proves out
Phase 3 — Extended	Returns, replenishment, reconciliation reporting, the finance interim account	33–49 days	Review after Phase 2
Total		151–237 days	

These are working-day estimates for development, testing and documentation, including project management and a 15% contingency; the cost section breaks them down. They exclude the contract day rate, which finance should apply. **Phase 1 alone delivers the stated objective.** Everything after it is improvement rather than repair.

Why build this rather than switch on NetSuite's own warehouse app

Warehouse Wizard already does three things NetSuite's warehouse module cannot do at all, and these are not opinions — each was stated by Wilson, Simplex's own NetSuite expert, during the meeting.

- **Pallet tracking.** NetSuite has no pallet record. Warehouse Wizard is built around pallets, with barcodes, dimensions, stacking and full traceability.
- **Replenishment at bin level.** NetSuite's reorder point works on a location as a whole, so it cannot see that a picking aisle has run dry while the bulk racking above it is full.
- **Visibility of stock in the picking area**, the specific gap Roget called "a huge red flag".

The honest framing is not that Warehouse Wizard is better than NetSuite. It is that **NetSuite is the right system of record and should stay that way, while Warehouse Wizard is the right system for the warehouse floor and can be shaped to Simplex's operation and vocabulary in ways NetSuite cannot.**

Delivered safely

A complete copy of Warehouse Wizard, application and database, will run as a beta sandbox connected to the NetSuite sandbox. The live system keeps running untouched. Nothing reaches production until Simplex has tested and signed off.

What is needed from Simplex now

Nothing large, but the project cannot be estimated precisely or started without them. Five items block everything else:

1. **The NetSuite security certificate must be uploaded.** The connection is built and currently stops at NetSuite's door. *(Jan)*
2. **Is Advanced Shipping switched on in NetSuite?** If it is off, every shipment Warehouse Wizard records would also automatically invoice the customer. *(Wilson)*
3. **Confirmation of how an inbound shipment receipt can be posted.** *(Jan)*
4. **What should happen when the container holds 95 and the purchase order says 100?** Raised in the meeting and left unresolved. *(Roget and Wilson)*
5. **Bins defined for Pine and Lower Estate, and mirrored into NetSuite.** Data entry, not development, but it gates Phase 1 testing. *(Wilson)*

What was decided in the meeting

This section records the decisions as captured. Participants should flag anything that misrepresents their position.

#	Decision	Who drove it	Status
D1	Receiving happens in Warehouse Wizard; NetSuite is updated from it. One entry, not two	Roget, agreed by Russell and Wilson	Agreed
D2	Three inbound document types in scope: purchase order, inbound shipment, transfer order	Wilson raised; Roget confirmed "Yes, those three"	Agreed
D3	Warehouse Wizard pulls the document on demand. The operator enters or scans the number from the printed receiving report and the lines load	Russell; Jan supported it on cost grounds	Agreed
D4	Jan will add the purchase order number to the printed receiving report, so there is something to scan	Jan volunteered	Agreed
D5	Pallet labels are printed at the container in small batches per product, not pre-printed	Russell, from prior experience	Agreed
D6	Cost and rate confirmation stays in NetSuite as a later task on the payables side. Warehouse Wizard does not touch pricing	Roget; Wilson confirmed rates are editable later	Agreed
D7	Warehouse Wizard records what was physically received; NetSuite holds the expectation	Russell	Agreed
D8	Pine and Lower Estate must be brought into Warehouse Wizard with bins defined	Roget opened the meeting with this	Agreed
D9	Bins will be created in NetSuite to mirror Warehouse Wizard, entered manually	Wilson	Agreed
D10	Visibility of the picking area will be switched back on	Roget: "I'm totally against it" (being off)	Agreed

#	Decision	Who drove it	Status
D11	Two distinct kinds of pick are needed: replenishment (bulk to picking area) and sales order (itemised)	Russell	Agreed in principle
D12	A beta sandbox copy will be built and connected to the NetSuite sandbox; production keeps running	Russell; Jan and Wilson both agreed	Agreed

Left open at the end of the meeting

Seven questions were raised and not closed. Each is answered or assigned later in this document: how to handle a quantity difference between expectation and delivery; whether a pallet can be represented in NetSuite at all; whether pallet numbers could serve as lot numbers; whether the finance interim account for picked stock is achievable; whether picking aisles should become storage bins; how reorder points should drive purchase orders; and how units of measure should work when an order does not make up whole cases.

Qualification of statements made

This is the check-and-double-check section. Statements made in good faith during a live meeting have been tested against Oracle's documentation and the actual Warehouse Wizard code. Two did not survive unchanged; both were remarks by Russell Hunte of Classic IT.

Statement	Said by	Verdict	Basis
An item receipt can't be created from scratch — it must come from a purchase order or inbound shipment	Wilson, confirmed by Jan	**Correct**	Oracle's documentation confirms receipts are created by transforming a source document
We can create item receipts as long as we link it to a PO	Jan	**Correct**	A standard, documented integration point

Statement	Said by	Verdict	Basis
If a PO is associated with an inbound shipment, the receipt can only be created via the inbound shipment	Wilson	**Correct, and consequential**	This is why the gap below matters. It removes the obvious fallback
Receiving one inbound shipment creates one item receipt per purchase order on it	Roget, demonstrated on screen	**Correct**	Matches Oracle's documentation
An inbound shipment can carry multiple purchase orders	Roget	**Correct**	Confirmed; the limit is 500 lines per shipment
There is no pallet record within NetSuite	Wilson	**Correct**	NetSuite has no pallet or licence-plate concept in its core product
Lot-numbered items are a different item record — you can't just add lot tracking to an existing item	Wilson	**Correct, and a significant constraint**	This makes the "pallet number as lot number" idea far more expensive than it first appears
In a wave there will be no GL posting — posting only happens at the item fulfilment	Wilson	**Correct**	This is why the interim account cannot be done the simple way
The reorder point is based on the location as a whole, not the bin	Wilson	**Correct**	A genuine NetSuite limitation, and an opportunity for Warehouse Wizard

Statement	Said by	Verdict	Basis
Polling NetSuite every five minutes would be a performance and licensing issue	Jan	**Correct, and the design has been changed to respect it**	NetSuite meters integration volume against an account-wide allowance that costs money to raise. The on-demand design in D3 avoids it
The picking-area functionality is pretty much all still there, it just needs hooking up	**Russell**	**Optimistic — corrected**	The data is ready: pallets already carry available, reserved, held and damaged quantities. But the picking logic deliberately takes whole pallets only, and this is written into the allocation rules rather than sitting behind a switch. Re-enabling it is a moderate rework of roughly 6–10 days, not a toggle. The estimates in this document reflect the corrected figure

Statement	Said by	Verdict	Basis
Warehouse Wizard is more robust and extendable than NetSuite	**Russell Hunte (Classic IT)**	**True in specific respects — must not be stated as a blanket claim**	Defensible and evidenced for pallet handling, bin-level replenishment, scan verification, picking-area visibility and configurability. Not defensible as a general statement: NetSuite holds inventory costing, the general ledger, procurement and financial control that Warehouse Wizard does not have and should never attempt to replicate. Stated broadly it would not hold. The comparison section sets out the qualified version
The upgrade will update all auto-sync databases and code to current code	**Russell Hunte (Classic IT)**	**Correct for code — dangerous if applied to data**	Code and database structure promote forward from beta to production. The beta database's contents must never be copied over production, because production will have accumulated real operational data during the beta period. Copying it back would destroy live records

Statement	Said by	Verdict	Basis
We're fully paperless when it comes to purchase orders	Roget	**Correct, with a useful exception**	A receiving report is printed just before the container is unstuffed. That printed sheet is the trigger for the whole receiving design in D3 and D4

What is possible — the direct answers

Each row answers a question raised in the meeting.

The question	Answer	Effort
Can Warehouse Wizard look up a purchase order and load its lines?	**Yes.** Standard, documented, nothing to install in NetSuite	Phase 1
Can Warehouse Wizard record the receipt so NetSuite marks the PO received?	**Yes.** Jan confirmed the mechanism during the meeting	Phase 1
Can Warehouse Wizard look up an inbound shipment and load its lines, across several POs?	**Yes.** Standard and documented	Phase 1
Can Warehouse Wizard complete the receipt of an inbound shipment?	**Yes, but only with a small piece of code deployed inside NetSuite by Jan.** This is the one genuine gap	Phase 1, plus 2–4 days of Jan's time
Can Warehouse Wizard receive a transfer order from another warehouse?	**Yes**, with one important rule: the receipt must match exactly what was shipped	Phase 1
Can Warehouse Wizard drive picking from a NetSuite sales order?	**Yes**	Phase 2

The question	Answer	Effort
Can picking be scan-verified instead of visual?	**Yes.** Warehouse Wizard already has barcode, label and printing infrastructure. Products without barcodes get labels printed at receipt	Phase 2
Can Warehouse Wizard handle an order for 20 items when stock is held in cases of six?	**Yes**, but this needs proper unit-of-measure support, which is substantial work	Phase 2
Can pallets be broken down and picked in part?	**Yes.** The data supports it now; the picking rules need rework	Phase 2
Can the contents of the picking area be seen?	**Yes.** Same piece of work	Phase 2
Can Warehouse Wizard receive customer returns?	**Yes**, provided NetSuite's Advanced Receiving feature is switched on, which must be confirmed	Phase 3
Can a pallet exist in NetSuite?	**Not as a pallet.** Three workarounds exist, all with real costs. Recommendation: keep pallets in Warehouse Wizard only	—
Can a GL entry be posted to an interim account when stock is picked?	**Not the way NetSuite does it natively** — NetSuite posts nothing until shipment. It can be done with a custom journal entry, but that is a finance decision rather than a technical one	Phase 3, optional
Can reorder points automatically raise purchase orders?	**Yes, and NetSuite should do this, not Warehouse Wizard.** But Warehouse Wizard can do what NetSuite cannot: trigger replenishment at bin level	Phase 3
Can this all be tested without risking the live system?	**Yes**	Phase 1

Receiving — the core of Phase 1

How it will work on the floor

1. The container arrives. The receiving report is printed from NetSuite as it is today, now carrying the purchase order number as well (Jan's change, D4).
2. The operator opens Receiving in Warehouse Wizard and scans or types the inbound shipment number, or the purchase order number for a local purchase.
3. Warehouse Wizard fetches the document from NetSuite live and displays every line: product, quantity expected, and which purchase order it belongs to.
4. The operator confirms the lines against the printed sheet.
5. As the container is unstuffed, the operator records what is actually received, product by product, and prints pallet labels in small batches as they go.
6. Lot numbers and expiry dates are captured where the product requires them.
7. Pallets are put away to bins.
8. Warehouse Wizard tells NetSuite what was received. NetSuite creates the item receipts, one per purchase order on the shipment, and updates the stock.
9. Rates and landed costs are confirmed later in NetSuite by the payables team.

Local purchases go straight from purchase order to receipt. Transfer orders from another Simplex warehouse follow the same path with the rules below.

The inbound shipment gap — the most important finding

What was assumed in the meeting. That because an item receipt can be created from a purchase order through the standard integration, the same would be true for an inbound shipment. Jan confirmed the purchase order path during the call and the discussion moved on.

What the documentation actually says. NetSuite treats "inbound shipment" and "receiving an inbound shipment" as two different things. The shipment can be created, read, updated and deleted through the standard integration, so Warehouse Wizard can look it up and load its lines without difficulty. But the act of receiving it uses a separate internal NetSuite record that Oracle documents only for scripts running inside NetSuite. There is no standard external equivalent.

Why the obvious workaround does not work. The natural fallback is to ignore the inbound shipment and receive each underlying purchase order instead. Wilson confirmed this during the meeting: "if we associate purchase order with inbound shipment, the item receipt will no longer be created via purchase order, we could only receive it via inbound shipment."

Why this matters commercially. Every imported container at Simplex uses an inbound shipment. If this surfaces during build rather than now, it appears late, at the worst possible moment, and the primary receiving path does not work.

The fix. Jan writes and deploys a small, purpose-built service inside NetSuite that accepts the received quantities from Warehouse Wizard and performs the receipt using NetSuite's internal mechanism. It is well-trodden work for a NetSuite developer and needs no NetSuite feature Simplex does not already have.

- **Owner:** Jan Roel
- **Estimated effort:** 2–4 days of NetSuite development
- **Schedule:** complete before Phase 1 acceptance testing
- **Interim:** Phase 1 can begin and progress on the purchase order and transfer order paths while this is built

Jan should confirm this analysis. If he finds a standard route this review missed, that is a welcome result and removes a risk — but the project must not be planned on the assumption that he will.

Transfer orders — one rule that changes the process

Receiving stock from another Simplex warehouse works, with a constraint Oracle states plainly: **the receipt must match the despatch exactly.** Same quantity, same lot and serial numbers, and it cannot be dated before the goods shipped.

So Warehouse Wizard cannot record a receipt of 48 against a despatch of 50. Either receive all 50 and post a separate stock adjustment for the 2 — clean, keeps NetSuite accurate, clear audit trail, and recommended — or stop and have the despatch record corrected first, which is accurate but blocks the warehouse.

Quantity differences — the unresolved question

Russell raised this and it was never settled: "if only 95 are received into the warehouse and NetSuite says 100, we have to figure out how we plan to resolve that."

There are three coherent policies. Simplex must pick one — the software can implement any of them, but not all three at once.

Policy	What happens	Best when
Receive what arrived	Warehouse Wizard records 95. NetSuite shows 95 received, 5 still outstanding	Short shipments are usually genuine and the balance may still arrive on a different shipment with the same PO.

Policy	What happens	Best when
Receive and flag	Warehouse Wizard records 95 and raises an exception for the buyer to close or chase	Differences need commercial follow-up with the supplier
Receive and adjust	Warehouse Wizard records 95 and closes the line, writing off the difference	Short deliveries are final and the paperwork should not stay open

Recommendation: receive and flag. It keeps the warehouse moving, keeps NetSuite truthful, and puts the commercial decision with the person who should make it. It is also the only one of the three that gives Roget the error visibility he asked for elsewhere in the meeting.

A related question: should Warehouse Wizard allow receiving more than ordered? NetSuite blocks this unless a specific preference is switched on, and Wilson needs to confirm the current setting.

The harder questions

These are the items where the meeting reached the edge of what was known.

Bringing Pine and Lower Estate in

Warehouse Wizard currently runs one warehouse. Its design already supports many, so this is not a rebuild — but every part of the system must be audited for assumptions that only one warehouse exists, and it needs real data work from Simplex: naming the warehouses, defining the zones, and defining every bin.

Wilson confirmed the NetSuite side already has locations for Pine and Lower Estate, but that bins must be created manually to mirror Warehouse Wizard. That mirroring is what allows stock to be tracked to a precise position in Warehouse Wizard while NetSuite holds the location-level total.

The important principle: a NetSuite **location** is a warehouse, and a NetSuite **bin** is a position within it. Oracle warns explicitly against creating NetSuite locations to represent areas inside a warehouse, because it breaks stock costing and reporting. The mapping Jan has already configured follows the correct pattern.

Units of measure — the biggest surprise in the meeting

Wilson raised something that had not been in scope: NetSuite holds products in multiple units. A bale contains six bags. An order can be placed in bales or in bags, and the unit used determines what comes out of stock.

Russell noted: "I honestly hadn't foreseen that. Due to what Kyden had said about not doing sales order picking inside of Wizard we had not foreseen having to be able to create a product either by eaches or by cases."

Warehouse Wizard today knows how many units are in a package and the package's dimensions, but it does not carry NetSuite's full unit hierarchy, and picking currently works in whole pallets, so the question has never arisen.

For sales order picking to work correctly this must be built properly. It affects receiving, picking, stock display, labels and reports. The awkward case is the one Russell described: an order for a quantity that does not make up whole cases. The system must switch to the smaller unit and make that visible to the operator rather than silently rounding. **This is a major piece of work and a prerequisite for sales order picking.** It cannot be skipped or approximated — getting it wrong means shipping the wrong quantities.

Pallets in NetSuite — recommendation: do not try

Wilson confirmed NetSuite has no pallet record. Three options were discussed.

Option	Assessment
A. Keep pallets in Warehouse Wizard only. NetSuite receives quantities per product per location; Warehouse Wizard holds the pallet detail	**Recommended.** No NetSuite change, no risk, no cost. Simplex keeps full pallet traceability where it is actually used — on the floor. This is precisely the division of labour that makes the two systems work together
B. Use the pallet number as a lot number (Russell's suggestion)	**Expensive, and only viable for some products.** Wilson identified the blocker: lot-tracked items are a different kind of item record in NetSuite. Existing products cannot simply have lot tracking switched on — the records must be recreated, with all the history and configuration that implies. Worth doing only for products that genuinely need expiry tracking

Option	Assessment
C. Create a custom pallet record in NetSuite (Jan's suggestion)	**Possible but of limited value.** It would be informational only and would not affect NetSuite's stock figures. It adds a second place to maintain pallet data without adding control

Recommendation: Option A, with Option B applied selectively to expiry-tracked products only. Wilson and Roget flagged this needs a separate discussion. That discussion should happen, but the default should be A.

The finance interim account — an honest answer

Roget asked for a GL entry when stock is picked, moving it out of inventory into an interim account, so finance can see in-process stock and catch errors: "if I at any moment see a value in that account, I know that something's going wrong in the process."

NetSuite will not do this natively. Wilson confirmed it — NetSuite posts nothing to the ledger until the item fulfilment. Moving stock into a staging area, whether by a wave or a bin transfer, has no financial effect at all.

There are two ways to get what Roget wants.

1. **Operational visibility without a GL entry.** Warehouse Wizard reports exactly what is in the picking area, by product, quantity and value, live. This gives the management information Roget described and costs very little, because it falls out of the picking-area work in Phase 2. It does not give him a ledger balance.
2. **A genuine GL entry.** Warehouse Wizard posts a journal entry into an interim account when stock is picked and reverses it on fulfilment. Achievable, but it means an external system writing to the general ledger, which needs the CFO's explicit approval, careful reversal handling and period-end discipline.

Recommendation: build option 1 in Phase 2 as a by-product of the picking work, then decide on option 2 once Roget has seen whether the report alone answers the need. Roget said "I'm not accepting it can't at the moment". It can be done; the open question is finance governance, not technical feasibility.

Reorder points and automatic purchase orders

Roget raised this at the very start: there is no minimum/maximum reordering today and he wants it. Wilson confirmed NetSuite already has this and can generate purchase orders from it, but only at location level, not bin level.

Recommendation — split it. Purchase ordering stays in NetSuite: it is a procurement and finance function, it already exists, and duplicating it would be wrong. Bin-level replenishment goes in Warehouse Wizard: noticing that a picking aisle has run dry while the bulk racking above it is full, and raising a task to move stock down. This is the thing NetSuite genuinely cannot do, and Warehouse Wizard already has the foundations for it.

What it costs

How to read the estimates

Effort is given in **working days** of development, including testing and documentation. The contract day rate is not included — finance should apply the agreed rate.

Three size bands are used, defined so the classification can be checked rather than taken on trust. **Minor** is contained within one screen or one rule, typically 1–3 days. **Moderate** touches several screens or an existing behaviour that must be reworked, typically 4–10 days. **Major** is a new capability or a change running across the whole system, 11 days or more.

The work packages

#	Work package	Band	Low	High	Phase
W1	Bring Pine and Lower Estate into Warehouse Wizard; audit single-warehouse assumptions	Moderate	5	8	1
W2	Look up purchase orders, inbound shipments and transfer orders from NetSuite on demand	**Major**	10	15	1
W3	Post receipts back to NetSuite across all three document paths	**Major**	12	18	1
W4	Record which NetSuite order line each Warehouse Wizard line came from	Minor–Moderate	2	4	1
W5	Map Warehouse Wizard bins to NetSuite bins, with tooling to create them	Moderate	4	7	1
W16	Integration reliability: queueing, ordering, retries, monitoring, operations runbook	Moderate	5	8	1
W15	Beta sandbox environment, parallel running and cutover	Moderate	6	10	1
	Jan's NetSuite-side work for inbound shipment receiving	*Moderate*	*2*	*4*	*1*
	Phase 1 subtotal (Warehouse Wizard)		**44**	**70**	
W6	Units of measure throughout	**Major**	12	18	2

#	Work package	Band	Low	High	Phase
W7	Partial pallets and picking-area visibility (rework)	Moderate	6	10	2
W8	Sales order picking driven from NetSuite, and despatch back to it	**Major**	15	22	2
W11	Transfer order despatch and receipt between warehouses	Moderate	6	10	2
W12	Scan-verified picking, and labels for products without barcodes	Moderate	5	8	2
	Phase 2 subtotal		**44**	**68**	
W9	Bin-level replenishment tasks	Moderate	6	10	3
W10	Returns: customer returns from NetSuite, and internal return-to-stock	Moderate–Major	8	12	3
W14	Nightly reconciliation and exception reporting between the two systems	Moderate	5	8	3
W13	Finance interim account (optional)	Moderate	5	8	3
	Phase 3 subtotal		**24**	**38**	
	Development subtotal		**112**	**176**	
	Project management, acceptance testing support, training materials (~20%)		22	35	
	Contingency (~15%, against the open questions)		17	26	
	Total		**151**	**237**	

Cost model

Total cost = total days × agreed day rate.

Scenario	Days	Cost at the agreed rate
Phase 1 only — solves the double-entry problem	60–96	
Phases 1 and 2 — adds full outbound operation	118–188	
All three phases	151–237	

Phase totals include a proportional share of project management and contingency.

What is deliberately excluded

So there are no surprises later, none of the following are in the numbers above.

- **Jan's NetSuite development time**, shown separately: roughly 2–4 days for the inbound shipment service, plus his time on the receiving report change, bin creation and sandbox support.
- **Simplex data entry** — defining zones and bins for Pine and Lower Estate, and creating the mirrored bins in NetSuite. This is significant clerical effort and should be resourced.
- **Any NetSuite licence changes**, should Simplex need additional integration capacity. The design is built to avoid this, but it would be Simplex's cost if it arises.
- **Recreating item records as lot-tracked**, should Simplex choose that route.
- **Hardware** — scanners, label printers and tablets for Pine and Lower Estate.
- **Ongoing support and hosting**, which stay under the existing arrangement.

The return

The case for Phase 1 does not rest on software features. It rests on three things. First, elimination of duplicate entry: every container received today is keyed twice, and Phase 1 removes the second keying entirely. Second, elimination of the reconciliation problem: two independent sets of numbers currently have to be made to agree, and after Phase 1 there is one entry feeding both, plus an automated report that flags any drift. Third, removal of a stock-accuracy risk that is currently invisible — Roget's "huge red flag" about the picking area is a real exposure and is not measurable today.

Simplex is better placed than Classic IT to put a figure on the hours currently spent on duplicate entry and reconciliation. That figure, against the Phase 1 cost, is the payback calculation.

Warehouse Wizard alongside NetSuite

Wilson suggested this comparison directly in the meeting: "if it will help, just compare Wizard with WMS."

The framing that matters

The question is not which system is better. NetSuite is the system of record and must stay that way — it holds stock valuation, the general ledger, procurement, purchasing and financial control, and nothing proposed here changes that or should.

The real question is: **what runs the warehouse floor?** And there, three facts confirmed by Simplex's own NetSuite expert during the meeting decide it.

Three things NetSuite's warehouse module cannot do

Capability	NetSuite	Warehouse Wizard	Source
Track pallets	No pallet record exists	Built around pallets: barcoded, with dimensions, stacking rules, geometry, reuse history and full traceability	Wilson, in the meeting
Replenish at bin level	Reorder points work on a location as a whole. It cannot see that a picking aisle is empty	Can trigger replenishment from a specific bin	Wilson, in the meeting
Show what is in the picking area	Nothing moves in the ledger until shipment; the staging step is invisible financially	Can report live contents and value of the picking area	Wilson, and Roget's "huge red flag"

These are not marketing claims. Each was stated by Wilson and each is corroborated by Oracle's own documentation.

Cost shape

	NetSuite's warehouse module	Warehouse Wizard
Licensing	Per user, recurring, rises as the warehouse team grows	Already owned. No per-user warehouse licence
Integration capacity	Heavy use consumes a metered allowance; more capacity costs more — exactly Jan's concern	The on-demand design deliberately minimises this
Changes	Configuration within what Oracle provides; anything beyond needs custom development inside NetSuite	Changes go where Simplex needs them, including terminology
Cost profile	Recurring operating cost	One-off capital build on an asset already owned

The extendability argument, stated properly

This point is set out precisely rather than broadly.

NetSuite is configured; Warehouse Wizard is built. That difference has a concrete consequence: **Warehouse Wizard can be shaped to how Simplex actually works and to the words Simplex actually uses.**

The meeting produced a perfect illustration. Russell raised that another client calls the picking aisle a "supermarket aisle", and immediately qualified it: "I wouldn't want to introduce supermarket unless other people are happy with it." That choice exists only because the system can be shaped. In NetSuite, a wave is a wave and a bin is a bin.

The same applies structurally. Roget's interim account, bin-level replenishment, pallet-level traceability, scan verification against Simplex's own labels — these are all things Warehouse Wizard can be extended to do because it is Simplex's software. In NetSuite they are either impossible or they become custom development inside a platform where custom development is expensive and constrained.

The proper claim is therefore: Warehouse Wizard is not a replacement for NetSuite and is not better than it. It is a warehouse execution system that is more adaptable than NetSuite's warehouse module, in the specific ways Simplex's operation needs, at a lower and less recurring cost — and it makes NetSuite more valuable by feeding it better data, faster, with less manual effort.

The claim is deliberately narrow; a broader one would not hold.

Delivery, risk and what Simplex must prepare

The beta sandbox

This section answers one question: can this be done without putting the live warehouse at risk? The answer is yes.

The live system is never the place where new work is proved. A complete, separate copy runs alongside it. Simplex tests on the copy. Only when Simplex signs off does anything reach the live system, and even then it can be reversed.

Component	Beta	Live during the beta
Application	Full copy at its own address	Untouched, running normally
Database	Separate database, its own data	Untouched, real operational data

Component	Beta	Live during the beta
NetSuite connection	Points at the **NetSuite sandbox**	Points at live NetSuite, as today
Credentials	Separate set — beta cannot reach live NetSuite	Unchanged
Users	Test accounts	Unchanged

The critical safeguard: the beta is given credentials that only work against the NetSuite sandbox. Even a serious error in beta cannot create a document in live NetSuite. This is not a procedural promise — it is enforced by the credentials themselves.

The stages

1. **Build the copy.** Application and database cloned, connected to the NetSuite sandbox with Jan.
2. **Develop and test.** Work packages built and tested against sandbox data. Simplex can watch progress at any point.
3. **Acceptance testing.** Wilson and the warehouse team run real scenarios — a real container, a real short delivery, a real transfer — against the sandbox. Nothing touches live.
4. **Sign-off.** Simplex confirms each work package. Anything that fails goes back to stage 2.
5. **Cutover.** Structural changes applied to the live database, then the application updated, in a quiet period outside operating hours.
6. **Watch.** The first live container is received with the team present and reconciliation checked immediately.

The upgrade — an important qualification

The intent as expressed was that the upgrade "will update all auto-sync databases and code to current code". That is right for code, and needs one correction for data.

Correct: the application code and the database structure move forward from beta to live. That is exactly how it should work, and the process is already established in this project.

Must not happen: the beta database's contents must never be copied over the live database. During the weeks the beta runs, the live system will be receiving real containers and recording real stock. Overwriting it with beta data would destroy that. The rule is: **structure and code move forward; data stays where it lives.**

The beta database is disposable. The live database is the only real one, and it is only ever altered, never replaced. Where existing live data needs reshaping to fit new structure, that is a controlled, reversible, tested migration, rehearsed on a copy of live data before it is run for real.

Every cutover has a documented way back, decided before it starts: what would trigger a rollback, who decides, and how long it takes. The live database is backed up immediately before any change.

Where the detail lives. The mechanics — environments, branching, migration order, credentials, seeding, deployment sequence — are operational detail that would not help a CFO and would clutter this document. That detail has been written as a separate technical runbook so whoever performs the work, including automated tooling, has an unambiguous procedure. This document carries the strategy and the safeguards; the runbook carries the steps.

Risks

Risk	Impact	Likelihood	Management
Inbound shipment receiving needs NetSuite-side code	Primary receiving path fails	**Confirmed** — a known requirement, not a possibility	Assigned to Jan, 2–4 days, scheduled before Phase 1 acceptance
Advanced Shipping is off in NetSuite	Every despatch Warehouse Wizard records would auto-invoice the customer	Unknown	Must be confirmed before Phase 2 is designed
Certificate not uploaded	Nothing can proceed	Live now	One administrative action in NetSuite
Bin data entry underestimated	Phase 1 testing delayed	Moderate	Size it early; it is clerical work that can start immediately
Units of measure more complex than described	Phase 2 estimate rises	Moderate	Wilson to supply real examples early
Short-delivery policy unresolved	Rework of the receiving screens	Moderate	Decide before W3 starts

Risk	Impact	Likelihood	Management
Integration capacity limits hit	NetSuite throttles; possible licence cost	Low, by design	On-demand lookup, low concurrency, monitored
Scope grows again	Cost and timeline drift	Moderate	Phase gates; each phase approved separately
NetSuite's warehouse module conflicts with Warehouse Wizard at the same location	Orders picked twice	Unknown	One system owns picking per location
Lot-tracked item conversion	Significant hidden data work	Moderate if that route is chosen	Recommendation is to avoid it

What Simplex must prepare

None of this is development work, but Phase 1 cannot be tested without it.

#	Task	Owner	Blocks
P1	Upload the security certificate in NetSuite to complete the connection	**Jan**	**Everything**
P2	Name and structure the Pine and Lower Estate warehouses: zones, aisles, bins	**Wilson / Roget**	W1, W5
P3	Create matching bins in NetSuite	**Wilson / Jan**	W5
P4	Confirm the NetSuite feature settings listed in Open questions	**Wilson**	Scope and estimates
P5	Provide sandbox access and a separate connection for the beta	**Jan**	W15
P6	Add the purchase order number to the printed receiving report	**Jan**	D4
P7	Decide the short-delivery policy	**Roget / Wilson**	W3
P8	Decide whether picking aisles become storage bins	**Roget**	Phase 2 design

#	Task	Owner	Blocks
P9	Identify which products genuinely need expiry and lot tracking	**Roget / Wilson**	Pallet decision
P10	Nominate who signs off acceptance testing	**Roget**	Cutover

Open questions

These are the questions the meeting raised but did not close, plus those this review has added. Each has an owner. The ones marked **blocking** prevent work starting or prevent the estimate from being fixed.

For Jan Roel — NetSuite development

#	Question	Why it matters
J1	**Can the security certificate be uploaded and the connection completed?**	**BLOCKING — everything.** The connection is built and tested as far as NetSuite's door. It is one administrative action in NetSuite
J2	**Can Jan confirm the inbound shipment receiving analysis and take on the NetSuite-side service?**	**BLOCKING Phase 1 acceptance.** The most important open item in this document
J3	Which general ledger account should stock adjustments post to, and the subsidiary if applicable?	Already outstanding; blocks any stock correction reaching NetSuite
J4	What integration capacity does the account have, and how much is already used?	Governs how hard the integration may run, and whether a licence cost arises
J5	Can Jan provide sandbox access and a **separate** connection and certificate for the beta?	The beta must not be able to reach live NetSuite
J6	If Warehouse Wizard receives against purchase orders rather than the inbound shipment, does the shipment's received quantity update?	Determines whether a fallback exists for J2

#	Question	Why it matters
J7	Jan offered to add the purchase order number to the printed receiving report — when can that be scheduled?	The whole receiving flow depends on there being something to scan
J8	Jan offered to test whether a pallet can be represented in NetSuite — what was the result?	The recommendation is not to, but the finding should be recorded
J9	If NetSuite's warehouse module is installed, can a despatch be recorded directly at one of its locations without going through a wave?	Determines whether Warehouse Wizard and NetSuite's module can coexist
J10	Can Jan confirm the exact field names for the receiving and despatch messages against the sandbox?	Prevents a class of avoidable failures during build

For Wilson Reyes — inventory and NetSuite configuration

#	Question	Why it matters
W1	**Is Advanced Receiving switched on?**	Determines the purchase order statuses, and whether customer returns can be received separately at all
W2	**Is Advanced Shipping switched on?**	**Financial risk.** If off, every despatch Warehouse Wizard records also invoices the customer
W3	Wilson indicated Simplex uses only Pick and Ship, not Pack — can this be confirmed?	Determines how many steps Warehouse Wizard reports back
W4	Are Advanced Bin Management and Inventory Status switched on?	Governs whether bin and status information can be sent at all
W5	Which products are lot or serial tracked today, and how many would need converting for expiry tracking?	Wilson flagged that conversion means recreating item records — the size of that matters
W6	Can Wilson supply real examples of multi-level units of measure — the rice bale/bag case and two or three others?	This is the largest Phase 2 estimate and real examples will sharpen it

#	Question	Why it matters
W7	Is "Allow Overage on Item Receipts" switched on?	Decides whether the warehouse may receive more than was ordered
W8	What should happen when 95 arrive against an order for 100?	Unresolved from the meeting. Recommendation: receive and flag
W9	For customer returns, who decides restock versus write-off — the warehouse or finance?	Determines whether that choice appears on the warehouse screen
W10	Do transfers between Pine, Lower Estate and the new warehouse use in-transit ownership, and is "Use Item Cost as Transfer Cost" set?	Changes how transfer receipts must be matched
W11	Should automatic purchase order generation from reorder points stay in NetSuite?	Recommendation: yes
W12	**Is NetSuite's warehouse module actually licensed and in use at any location?**	If it is, one system must own picking at each location, or orders get picked twice

For Roget Williams — policy and commercial

#	Question	Why it matters
R1	**Confirm the short-delivery policy**	Shapes the receiving screens. Recommendation: receive and flag
R2	Is the interim GL account a firm requirement, or would a live picking-area report satisfy it?	Decides whether W13 is in scope. Recommendation: see the report first
R3	Should the picking aisles become storage bins, with a dedicated staging area by the doors?	Roget raised this as a pushback. It changes the Phase 2 design and should be settled before Phase 2
R4	Which products genuinely need expiry tracking?	Wilson has flagged that this carries real cost
R5	Which phases are approved, and in what order?	See the cost section

#	Question	Why it matters
R6	Who signs off acceptance testing, and who authorises cutover?	See delivery

Recommendation

1. **Answer the five blocking questions** — J1, J2, W1, W2 and R1. None requires development. They can be closed in a single follow-up meeting.
2. **Approve Phase 1.** It solves the problem that prompted this work, it is the smallest useful increment, and it proves the connection under real conditions before anything larger is committed.
3. **Start the data preparation now** — P2 and P3. Defining bins for Pine and Lower Estate does not depend on any software being finished, and it is the most likely thing to delay Phase 1 if left late.
4. **Schedule Jan's NetSuite-side work early**, not late. It is small, but Phase 1 cannot be accepted without it.
5. **Approve Phase 2 in principle and revisit after Phase 1 goes live.** Phase 1 will teach Simplex things about the integration that will make the Phase 2 estimates better.
6. **Defer Phase 3.** Returns, replenishment and the interim account are all genuinely valuable, and none is urgent.

Notes for the NetSuite developer

For Jan only. Nothing here changes anything above.

Integration points. All standard NetSuite REST record endpoints unless noted. Receiving a purchase order, a transfer order or a customer return each transform the source document into an item receipt. Despatching a transfer order or a sales order transforms it into an item fulfilment. Reading any of them is a standard record read plus SuiteQL for list and search. **Receiving an inbound shipment has no transform** — Oracle documents the `receiveinboundshipment` record for SuiteScript only, and that is the gap described earlier.

Suggested contract for the inbound shipment service. In: the inbound shipment internal id; a list of lines each with the shipment line key, quantity received, receiving location, and any lot, serial, bin or status detail; plus a Warehouse Wizard reference for idempotency. Out: the internal ids of the item receipts NetSuite created, one per purchase order. Mechanism: load `receiveinboundshipment` by the shipment id, set `quantitytobereceived` on the relevant `receiveitems` lines, apply inventory detail, save.

The Warehouse Wizard reference should be stored on the created records so a retry after a timeout can be detected rather than duplicated — Warehouse Wizard already uses this pattern for stock adjustments.

Points to verify against the sandbox. Line identification on item receipt transforms, and whether `orderLine` or `line` is expected; always read the source document's lines first, because line numbers are not sequential. Inbound shipment lines reference the purchase order's line unique key, not the item's internal id. Field names and casing for inventory detail on each record, best taken from the account's own metadata rather than from documentation. Whether a transfer order receipt can nominate which fulfilment it belongs to through the standard endpoint, or whether that also needs a script. Status letter codes for each transaction type in this account.

Existing Warehouse Wizard behaviour. Authentication is machine-to-machine with a signed assertion; the certificate is generated and awaiting upload. There is an existing job queue with atomic claiming, retry, dead-lettering and payload logging, which new document types extend rather than replace. Duplicate protection uses a Warehouse Wizard reference written onto the NetSuite record, with a lookup by that reference after a failure or timeout. Inbound messages from NetSuite are already accepted and parked for purchase orders, sales orders, transfer orders and fulfilments; the receivers do not exist yet and this proposal builds them. The background drain runs on a schedule outside the database, because this database has no facility to call out directly.

A full documentation review with verified source links, and a read-only discovery pack of queries that can answer many of the open questions automatically once J1 is closed, is held alongside this proposal.

Glossary

Term	Meaning
Purchase order	The order placed on a supplier. Created in NetSuite
Inbound shipment	A NetSuite document representing a physical shipment, typically a container, which can carry lines from several purchase orders
Item receipt	The NetSuite document that records goods arriving and increases stock
Item fulfilment	The NetSuite document that records goods leaving and decreases stock

Term	Meaning
Transfer order	A movement of stock between two Simplex warehouses
Return authorisation (RMA)	Authorisation for a customer to return goods
Location (NetSuite)	A warehouse. Pine, Lower Estate and the new warehouse are each a location
Bin (NetSuite)	A position inside a warehouse. Equivalent to a location in Warehouse Wizard
Lot number	A batch identifier, usually with an expiry date, tracked by NetSuite for products configured for it
Wave	NetSuite's mechanism for releasing orders to its own warehouse app. Not used in this proposal
Advanced Receiving	A NetSuite setting that separates receiving goods from paying for them
Advanced Shipping	A NetSuite setting that separates shipping goods from invoicing for them
Transform	NetSuite's term for creating one document from another — a receipt from a purchase order, for example
Sandbox	A complete copy of a system used for testing, where mistakes are harmless
Cutover	The moment tested changes are applied to the live system

---

Every factual claim about NetSuite behaviour in this document is either sourced from Oracle's published documentation, stated by Wilson Reyes or Jan Roel in the meeting of 21 September 2026, or explicitly marked as unverified. Claims that could not be verified are listed in Open questions with an owner, and the cost estimates carry contingency for them.